How to build Conda packages?

Romain Beucher[1]

[1] Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193440

Keywords Tricks of the Trade, Documentation, #editors-pick

As mentioned, packages can be installed directly from our conda channel. You may however want to build your own package at some point in the future or you may want to fix some of ours by submitting a Pull Request on github (You are more welcome to do so...)

PUBLISHED Nov 23, 2020
ARTICLE UWTN 2020-012
VERSION 1.0.0
LICENCE CC-BY-4.0
ARCHIVED 2026-08-10T04:19:43Z
SOURCE https://www.underworldcode.org/build-conda-packages/

The following goes through the steps of building a conda package for Badlands which contains Python and Fortran code. This is relatively advanced but not really difficult.

0.a. Installing and Updating `conda-build`

To install conda-build, in your terminal window or an Anaconda Prompt, run:

```
conda install conda-build
```

1. A Conda recipe for Badlands

conda build requires a conda recipe which is usually a combination of files such as:

- A meta.yaml file
- A build script called `build.sh` for Linux and MacOS and `build.bat` for Windows.
- Sometimes a `yum_requirements.txt` file for some additional libraries that are not available on conda-forge.
- A LICENSE file.

The Badlands conda recipe is relatively simple and contains only a `meta.yaml` file and the License file.

1.a. The meta.yaml file

conda recipes use a `meta.yaml` file. The YAML (a recursive acronym for "YAML Ain't Markup Language") format is a text format that is meant to be easily readable. A cheat sheet and full specification are available at the official site. The format is organised around to main structures:

- Dictionaries:

```
package:
  name: name
  version: version
```

- Lists:

```
host:
  - python
  - pytriangle
  - numpy >=1.15.0
  - pip
```

The YAML structures can be completed with some Jinja2 code which allows to declare variables and access them:

```
{% set name = "badlands" %}
{% set version = "2.0.25" %}

package:
  name: {{ name|lower }}
  version: {{ version }}
```

Jinja2 is a very powerful templating language for Python. It is definitely worth checking out if you want to automated / simplify your config files.

1.b. Basic structure of the recipe

The `meta.yaml` recipe is organised into a list of structures that must be provided to `conda build`. They are:

- The **package** dictionary which must contain the **name** and the **version** number of the package.
- The **source** dictionary which contains the **url** (which can be a local path or an http address). Note that providing a checksum such as sha256 is considered good practice. It can easily be generated on a MacOs or Linux system using the following command: `openssl sha256 my_tarball.tar.gz`
- The **build** dictionary that contains the build **number** that you as a maintener are responsible to increment and the **build** instructions on how to build the package. In the case of *Badlands*, the instructions are fairly simple and rely on a `pip install` method. You could provide a `build.sh` file with more complex instructions. I recommend you check the *Lavavu* recipe for an example of a more complex set of build instructions. Note that in its current form the *Badlands* conda recipe skips Windows system as we chose to rely on the Windows Subsystem for Linux (WSL) available on Windows 10.
- The requirement dictionary that contains 3 lists of dependencies for the **build**, **host** and **run** systems. The **build** section should only contains the tools that are required to build the package. This generally include the compilers and some configuration tools such as *make* or *cmake* etc. The **host** section contains a list of packages that need to be specific to the target platform, when the target platform is not necessarily the same as the build platform. Shared libraries should be listed here. The Python interpreter must also be listed here. The **run** section contains the dependencies that are needed when running a program. Typically all the packages that are imported by the package we are building will need to be listed here.

Note: Version constrains can be easily set within the lists of dependencies. Conda will use those constrain to resolve the dependencies when creating a new environment.

- The **test** section which contains the test that conda build should run once the package has been built. This section is optional but highly recommended. For a Python package, it includes a list of imports. It can also run unit tests etc.
- The **about** section contains a list of metadata that describe the package. It usually includes a link to the **home**page of the project, the **license**, a link to the full **license_file**, a **summary** and a link to the documentation, **doc_url**.
- The **extra** section contains some extra metadata that might be useful such as a list of the maintainers names.

1.c. The complete recipe

Badlands requires a Fortran and a C compiler. Anaconda provides a list of default compilers that can be accessed using the {{compiler()}}
expression. It is the recommended way to do things but you could also specify a specific compilers. You could also provide a list of variants compilers but this is beyond the scope of this blog post.

```
{% set name = "badlands" %}
{% set version = "2.0.25" %}

package:
  name: {{ name|lower }}
  version: {{ version }}
```

```
source:
  url: "https://pypi.io/packages/source/{{ name[0] }}/{{ name }}/{{ name }}-{{ version }}.tar.gz"
  sha256: 339175849a62412e6e445be40b1c392ad1194b7c97746492010e7d3e26045814

build:
  skip: true # [win]
  number: 0
  script: {{ PYTHON }} -m pip install badlands/ -vv

requirements:
  build:
    - {{ compiler('fortran') }}
    - {{ compiler('c') }}
  host:
    - python
    - pytriangle
    - numpy
    - pip
  run:
    - gflex
    - h5py
    - matplotlib
    - meshplex
    - numpy
    - pandas
    - python
    - scikit-image
    - scipy
    - six
    - pytriangle

test:
  imports:
    - badlands
    - badlands.flow
    - badlands.forcing
    - badlands.hillslope
    - badlands.simulation
    - badlands.surface
    - badlands.underland

about:
  home: "https://github.com/badlands-model"
  license: GPL-3.0+
  license_family: GPL
  license_file: LICENSE
  summary: "Basin and Landscape Dynamics (Badlands) is a TIN-based landscape evolution model"
  doc_url: https://badlands.readthedocs.io/

extra:
  recipe-maintainers:
    - rbeucher
    - tristan-salles
```

2. The conda build process

Once ready you can test that the package builds using:

```
conda build -c conda-forge -c geo-down-under recipe/
```

where `recipe` is the folder that contains the full set of files of the conda recipe.

Here I copy the steps performed by conda-build as described in the official documentation:

Conda-build performs the following steps:

1. Reads the metadata.
2. Downloads the source into a cache.
3. Extracts the source into the source directory.
4. Applies any patches.
5. Re-evaluates the metadata, if source is necessary to fill any metadata values.
6. Creates a build environment and then installs the build dependencies there.
7. Runs the build script. The current working directory is the source directory with environment variables set. The build script installs into the build environment.
8. Performs some necessary post-processing steps, such as shebang and rpath.
9. Creates a conda package containing all the files in the build environment that are new from step 5, along with the necessary conda package metadata.
10. Tests the new conda package if the recipe includes tests:
11. Deletes the build environment and source directory to ensure that the new conda package does not inadvertantly depend on artifacts not included in the package.
12. Creates a test environment with the package and its dependencies.
13. Runs the test scripts.